Министерство просвещения Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Тульский государственный педагогический
университет им. Л.Н. Толстого»
(ФГБОУ ВО «ТГПУ им. Л.Н. Толстого»)

Институт передовых информационных технологий

КУРСОВАЯ РАБОТА

по дисциплине «Методы и технология программирования»

на тему:

Реализация технологий и методов обеспечения информационной безопасности пользователей в мессенджерах

Выполнила:
студентка 2 курса группы 1520441
института передовых информационных
технологий
направления «Математическое
обеспечение и администрирование
информационных систем»
Направленности (профиля) технологии
программирования и анализ данных
Корнетов Виталий Отчество

Руководитель:
к.п.н., доцент
Даниленко Софья Валерьевна

Тула 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ПРОТОКОЛОВ БЕЗОПАСНОСТИ ДЛЯ МЕССЕНДЖЕРА 6

1.1 Анализ предметной области 6

1.2 Протокол TLS как средство защиты сетевого взаимодействия 10

1.3 Сравнительный анализ существующих решений 16

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ЗАЩИЩЁННОГО МЕССЕНДЖЕРА 21

2.1 Алгоритмы работы программного комплекса 21

2.2 Программная реализация системы 24

2.3 Тестирование программного комплекса 27

ЗАКЛЮЧЕНИЕ 34

СПИСОК ЛИТЕРАТУРЫ 35

ПРИЛОЖЕНИЯ 37

ВВЕДЕНИЕ

В условиях стремительного развития цифровых технологий обмен информацией посредством сети Интернет стал неотъемлемой частью повседневной деятельности человека. Электронная коммуникация используется в образовательной сфере, государственном управлении, бизнесе, банковском секторе и повседневном общении граждан. Одним из наиболее востребованных средств коммуникации являются мессенджеры, обеспечивающие обмен сообщениями в режиме реального времени.

Актуальность темы исследования обусловлена постоянным ростом количества пользователей сетевых сервисов и увеличением объёмов передаваемой информации. Согласно данным Федеральной службы государственной статистики, уровень проникновения сети Интернет в Российской Федерации ежегодно увеличивается, а цифровые сервисы становятся основным инструментом взаимодействия между пользователями [20]. Дополнительные исследования ВЦИОМ и Mediascope показывают устойчивый рост использования мобильных приложений для обмена сообщениями и удалённого взаимодействия [15, 5].

Увеличение количества пользователей сетевых сервисов приводит к росту объёмов передаваемых данных и, как следствие, к повышению интереса злоумышленников к их перехвату. В настоящее время значительную угрозу представляют атаки, направленные на нарушение конфиденциальности, целостности и доступности информации [23]. Наиболее распространёнными являются атаки типа «Man-In-The-Middle», перехват сетевого трафика, подмена серверов, а также распределённые атаки отказа в обслуживании (DDoS) [13].

Необходимость разработки программных средств защиты информации подтверждается рядом нормативных документов Российской Федерации [16]. В соответствии с положениями Доктрины информационной безопасности Российской Федерации, утверждённой Указом Президента Российской Федерации №646 от 5 декабря 2016 года, обеспечение безопасности

информационной инфраструктуры и защита передаваемых данных являются приоритетными направлениями развития информационных технологий. Кроме того, вопросы обеспечения информационной безопасности отражены в Стратегии развития информационного общества в Российской Федерации на 2017–2030 годы, утверждённой Указом Президента Российской Федерации №203 от 9 мая 2017 года [19].

Но несмотря на наличие большого количества мессенджеров как зарубежной, так и отечественной разработки. К наиболее известным зарубежным решениям относятся Signal и WhatsApp, а среди отечественных разработок можно выделить Telegram, VK Мессенджер, ТамТам, МАКС и ряд корпоративных платформ обмена сообщениями.

Большинство современных мессенджеров используют защищённые протоколы передачи данных, основанные на технологиях TLS и современных криптографических алгоритмах [11]. Это позволяет обеспечивать конфиденциальность передаваемой информации и защиту от большинства распространённых сетевых атак [7].

Однако наличие механизмов шифрования не гарантирует абсолютной безопасности программного продукта [1]. Исследователи в области информационной безопасности регулярно выявляют новые уязвимости, связанные с ошибками реализации, настройкой серверной инфраструктуры, управлением криптографическими ключами и особенностями архитектуры приложений. Кроме того, значительная часть существующих решений является закрытым программным обеспечением, что затрудняет изучение принципов их функционирования и механизмов обеспечения безопасности.

Также следует учитывать, что многие современные мессенджеры ориентированы прежде всего на массовое использование и удобство пользователей, тогда как процессы установления защищённого соединения, TLS Handshake, проверки сертификатов, журналирования событий безопасности и мониторинга сетевой активности скрыты от конечного пользователя [7]. В результате становится затруднительным изучение

принципов работы современных механизмов защиты информации на практическом уровне.

В связи с этим актуальной задачей является разработка учебного программного комплекса, позволяющего наглядно продемонстрировать процессы установления защищённого TLS-соединения, шифрования сообщений, мониторинга сетевого трафика и обнаружения аномальной активности в компьютерных сетях [18].

Целью курсовой работы является разработка защищённого мессенджера, обеспечивающего безопасную передачу сообщений между клиентом и сервером посредством протокола TLS с возможностью мониторинга сетевой активности.

Программа будет реализовывать следующие функции:

– демонстрировать механизм установления TLS-соединения;

– демонстрировать обмен сообщениями между клиентом и сервером;

– демонстрировать систему журналирования событий безопасности;

– содержать модуль мониторинга сетевого трафика и обнаружения аномальной активности.

Помимо этого, программа будет протестирована по принципам Белого и чёрного ящика.

Объектом исследования является процесс защищённого обмена сообщениями в компьютерных сетях.

Предметом исследования являются методы и средства программной реализации безопасного сетевого взаимодействия с использованием протокола TLS.

При разработке системы учитывается опыт существующих отечественных платформ обмена сообщениями, в том числе мессенджера МАКС. Разрабатываемый прототип призван заложить основу для создания нового российского общедоступного мессенджера, в архитектуре которого особое внимание будет уделено вопросам информационной безопасности, защищённого обмена данными и устойчивости к современным киберугрозам.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ПРОТОКОЛОВ БЕЗОПАСНОСТИ ДЛЯ МЕССЕНДЖЕРА

1.1 Анализ предметной области

Рассмотрим современные угрозы информационной безопасности в системах обмена сообщениями.

Информационная безопасность представляет собой состояние защищённости информации от несанкционированного доступа, изменения, уничтожения или блокирования [23]. В современных условиях развитие сетевых технологий сопровождается постоянным ростом количества угроз безопасности.

Одной из наиболее распространённых угроз является перехват сетевого трафика. Если передача данных осуществляется без использования механизмов шифрования, злоумышленник получает возможность анализировать содержимое пакетов и получать доступ к конфиденциальной информации [21].

Особую опасность представляют атаки типа Man-In-The-Middle. В ходе такой атаки злоумышленник располагается между клиентом и сервером и получает возможность перехватывать, изменять или подменять передаваемые данные [13].

Другой распространённой угрозой является подмена серверов [21]. Пользователь может быть перенаправлен на поддельный сервер, внешне не отличающийся от оригинального. В результате злоумышленник получает возможность похищать учётные данные пользователей и конфиденциальную информацию.

Серьёзную угрозу представляют DDoS-атаки, целью которых является нарушение доступности информационной системы [23]. При реализации подобных атак большое количество устройств одновременно направляет запросы на сервер, перегружая вычислительные ресурсы системы.

Кроме технических угроз существуют угрозы, связанные с человеческим фактором [10]. Пользователи могут самостоятельно

раскрывать конфиденциальную информацию вследствие применения методов социальной инженерии.

Таким образом, современные системы обмена сообщениями должны обеспечивать комплексную защиту от широкого спектра угроз информационной безопасности.

Для обеспечения безопасности сетевого взаимодействия используются различные методы криптографической защиты информации [11].

Основными направлениями защиты являются:

– шифрование данных;

– контроль целостности информации;

– аутентификация участников взаимодействия;

– управление криптографическими ключами;

– использование цифровых сертификатов.

Современные криптографические системы основаны на использовании математических алгоритмов, обеспечивающих невозможность получения исходной информации без знания специальных ключей.

Симметричное шифрование предполагает использование одного ключа как для шифрования, так и для расшифрования данных.

Симметричные алгоритмы шифрования широко применяются в современных информационных системах благодаря высокой производительности и эффективности обработки больших объёмов данных. В настоящее время наиболее распространённым алгоритмом является AES (Advanced Encryption Standard), рекомендованный Национальным институтом стандартов и технологий США (NIST) для защиты конфиденциальной информации [11].

Преимуществами симметричного шифрования являются высокая скорость работы и низкие вычислительные затраты.

Наиболее распространёнными алгоритмами данного типа являются:

– AES;

– DES;

– Triple DES;

– ChaCha20.

Несмотря на высокую производительность, симметричное шифрование обладает существенным недостатком, связанным с необходимостью безопасной передачи секретного ключа между участниками взаимодействия.

Асимметричная криптография использует два различных ключа: открытый и закрытый.

Основы современной асимметричной криптографии были заложены в работах Диффи и Хеллмана, предложивших концепцию обмена ключами через открытый канал связи [2]. В дальнейшем данная концепция стала основой большинства современных протоколов безопасного взаимодействия.

Открытый ключ может распространяться свободно и использоваться для шифрования информации, тогда как закрытый ключ хранится в секрете владельцем и применяется для расшифрования данных.

К преимуществам данного подхода относятся:

– безопасное распределение ключей;

– возможность использования цифровых подписей;

– высокий уровень безопасности.

Наиболее распространёнными алгоритмами асимметричной криптографии являются RSA, Diffie-Hellman и алгоритмы на основе эллиптических кривых.

В современных системах асимметричное шифрование обычно применяется совместно с симметричным для повышения производительности.

Одной из важнейших задач обеспечения безопасности сетевого взаимодействия является подтверждение подлинности участников обмена информацией. Для решения данной задачи применяется инфраструктура открытых ключей (Public Key Infrastructure, PKI) [3], основанная на использовании цифровых сертификатов.

Цифровой сертификат представляет собой электронный документ, подтверждающий принадлежность открытого криптографического ключа конкретному субъекту. В качестве субъекта могут выступать физические лица, организации, программные системы, серверы или сетевые сервисы.

Наиболее распространённым стандартом цифровых сертификатов является стандарт X.509. Сертификаты данного формата используются в большинстве современных систем защищённого взаимодействия, включая веб-сайты, банковские системы, облачные сервисы и мессенджеры.

Стандартный сертификат X.509 содержит следующие основные сведения:

- имя владельца сертификата;
- открытый ключ владельца;
- срок действия сертификата;
- наименование удостоверяющего центра;
- алгоритм подписи;
- цифровую подпись удостоверяющего центра.

Использование цифровых сертификатов позволяет решить проблему подтверждения подлинности сервера и предотвратить подмену узлов злоумышленниками.

Процесс проверки сертификата включает несколько этапов [3]. На первом этапе осуществляется проверка цифровой подписи удостоверяющего центра. Затем проверяется срок действия сертификата. После этого выполняется сопоставление имени сервера с информацией, указанной в сертификате.

Если хотя бы одна из проверок завершается ошибкой, соединение считается небезопасным и устанавливать его не рекомендуется.

Особую роль в инфраструктуре открытых ключей играют центры сертификации (Certificate Authority, CA) [3]. Центр сертификации является доверенной организацией, осуществляющей выпуск, регистрацию и подтверждение подлинности цифровых сертификатов.

В глобальной сети Интернет функционирует большое количество удостоверяющих центров, сертификаты которых предустановлены в операционных системах и веб-браузерах. Среди наиболее известных международных удостоверяющих центров можно выделить DigiCert, GlobalSign, Sectigo и Let's Encrypt.

Для корпоративных сетей и исследовательских проектов нередко используются собственные центры сертификации. Такой подход позволяет создавать внутреннюю инфраструктуру доверия и выпускать сертификаты для собственных сервисов.

В рамках данной курсовой работы для демонстрации работы защищённого соединения используется собственный локальный центр сертификации, выпускающий сертификат сервера. Это позволяет на практике изучить процессы генерации ключей, подписания сертификатов и проверки подлинности участников взаимодействия.

Использование цифровых сертификатов является обязательным элементом большинства современных систем информационной безопасности и обеспечивает высокий уровень доверия между участниками сетевого взаимодействия.

1.2 Протокол TLS как средство защиты сетевого взаимодействия

Одним из наиболее распространённых механизмов защиты информации при передаче данных через компьютерные сети является протокол TLS (Transport Layer Security).

Протокол TLS является международным стандартом защиты сетевого взаимодействия и применяется в большинстве современных веб-сервисов, облачных платформ и систем обмена сообщениями. Актуальная версия протокола TLS 1.3 была стандартизирована организацией IETF в документе RFC 8446 [7].

TLS представляет собой криптографический протокол, предназначенный для обеспечения конфиденциальности, целостности и подлинности передаваемой информации. Он является развитием более раннего протокола SSL (Secure Sockets Layer), который в настоящее время считается устаревшим и не рекомендуется к использованию [11].

Основной задачей TLS является создание защищённого канала связи между клиентом и сервером через потенциально небезопасную сеть.

На сегодняшний день протокол TLS применяется практически во всех современных сетевых сервисах:

- веб-сайтах;
- интернет-банкинге;
- облачных платформах;
- корпоративных системах;
- мессенджерах;
- почтовых сервисах;
- системах электронного документооборота.

Использование TLS позволяет предотвратить перехват передаваемых данных и значительно усложняет проведение атак типа Man-In-The-Middle.

Основными свойствами TLS являются:

Конфиденциальность

Конфиденциальность достигается посредством шифрования данных. Даже если злоумышленник получит доступ к сетевому трафику, он не сможет прочитать содержимое сообщений без соответствующих криптографических ключей.

Целостность

Целостность обеспечивает невозможность незаметного изменения информации в процессе передачи. Для этого используются криптографические хэш-функции и специальные механизмы контроля целостности.

Аутентификация

Аутентификация позволяет клиенту убедиться в том, что соединение устанавливается именно с тем сервером, с которым оно должно быть установлено. Данная задача решается посредством цифровых сертификатов.

Современные версии TLS используют широкий набор криптографических алгоритмов. Как правило, для обмена ключами применяются алгоритмы асимметричной криптографии, а для непосредственного шифрования данных используются высокопроизводительные симметричные алгоритмы.

Такой подход позволяет одновременно обеспечить высокий уровень безопасности и производительности системы.

Принципы работы TLS. Работа TLS основана на комбинации нескольких криптографических механизмов.

На начальном этапе клиент и сервер выполняют процедуру согласования параметров безопасности. В процессе данного взаимодействия определяются:

- версия протокола TLS;
- используемые криптографические алгоритмы;
- сертификат сервера;
- параметры обмена ключами.

После завершения процедуры согласования формируется общий секретный ключ, который в дальнейшем используется для симметричного шифрования передаваемых данных.

Использование симметричного шифрования на этапе обмена сообщениями обусловлено высокой производительностью подобных алгоритмов. Это позволяет обеспечивать защиту информации без существенного снижения скорости передачи данных.

Современные версии TLS используют такие алгоритмы, как:

- AES-128;
- AES-256;
- ChaCha20-Poly1305.

Для контроля целостности данных применяются криптографические хэш-функции семейства SHA-2 и SHA-3.

Благодаря сочетанию различных криптографических механизмов TLS обеспечивает высокий уровень защиты информации и является фактическим стандартом безопасного сетевого взаимодействия.

TLS Handshake. Наиболее важным этапом работы протокола TLS является процедура TLS Handshake.

TLS Handshake представляет собой последовательность сообщений между клиентом и сервером, предназначенную для установления защищённого соединения.

Процедура TLS Handshake обеспечивает согласование криптографических параметров между клиентом и сервером, проверку цифровых сертификатов и формирование общего сеансового ключа, который в дальнейшем используется для симметричного шифрования передаваемых данных.

Процесс TLS Handshake включает следующие этапы:

Этап 1. Client Hello

Клиент инициирует соединение и отправляет серверу сообщение Client Hello.

Данное сообщение содержит:

- поддерживаемые версии TLS;
- список криптографических алгоритмов;
- случайное число клиента;
- дополнительные параметры безопасности.

Этап 2. Server Hello

Сервер отвечает сообщением Server Hello.

В данном сообщении содержатся:

- выбранная версия TLS;
- выбранный набор криптографических алгоритмов;
- случайное число сервера;

– сертификат сервера.

Этап 3. Проверка сертификата

Клиент проверяет полученный сертификат.

Проверяются:

– подпись центра сертификации;

– срок действия сертификата;

– имя сервера;

– цепочка доверия.

При обнаружении ошибок соединение разрывается.

Этап 4. Обмен ключами

После успешной проверки сертификата выполняется обмен криптографическими параметрами.

В современных реализациях TLS наиболее часто применяется алгоритм ECDHE (Elliptic Curve Diffie-Hellman Ephemeral), обеспечивающий прямую секретность (Forward Secrecy).

Этап 5. Формирование сеансовых ключей

На основе обменённых параметров обе стороны вычисляют одинаковый секретный ключ.

Этот ключ никогда не передаётся по сети в открытом виде.

Этап 6. Завершение Handshake

После формирования ключей стороны подтверждают успешность процедуры и переходят к защищённому обмену данными.

С этого момента весь сетевой трафик шифруется.

Именно процедура TLS Handshake реализуется в разработанном программном комплексе и является одним из ключевых объектов исследования в рамках данной курсовой работы.

Использование сертификатов X.509 в TLS

Одним из ключевых механизмов обеспечения безопасности в протоколе TLS является использование цифровых сертификатов стандарта X.509. Данный стандарт был разработан Международным союзом

электросвязи (ITU-T) и в настоящее время является основой инфраструктуры открытых ключей большинства информационных систем.

Основной задачей сертификата является подтверждение принадлежности открытого криптографического ключа конкретному владельцу. Благодаря этому клиент получает возможность убедиться в подлинности удалённого сервера и исключить возможность подключения к поддельному узлу.

В процессе установления TLS-соединения сервер передаёт клиенту собственный сертификат. Клиент выполняет ряд проверок, позволяющих определить возможность доверять полученному сертификату.

К основным проверкам относятся:

- проверка цифровой подписи удостоверяющего центра;
- проверка срока действия сертификата;
- проверка цепочки доверия;
- проверка имени узла;
- проверка расширений сертификата.

Одним из наиболее важных этапов является проверка имени узла (Hostname Verification). Клиент сравнивает адрес сервера, к которому осуществляется подключение, со значением поля Subject Alternative Name, содержащимся в сертификате.

Если адреса не совпадают, TLS-соединение считается потенциально опасным и разрывается.

В современных информационных системах сертификаты X.509 используются практически повсеместно. Они применяются для защиты веб-сайтов, электронной почты, VPN-сервисов, облачных платформ и систем обмена сообщениями.

В разработанном программном комплексе цифровые сертификаты используются для подтверждения подлинности сервера и организации безопасного TLS-соединения между клиентом и сервером.

Использование сертификатов позволяет продемонстрировать принципы работы современной инфраструктуры открытых ключей и механизмы формирования доверенных соединений.

1.3 Сравнительный анализ существующих решений

Современный рынок программных средств обмена сообщениями представлен большим количеством приложений как зарубежной, так и отечественной разработки.

Большинство современных мессенджеров используют различные механизмы криптографической защиты информации, включая протоколы TLS, сквозное шифрование, цифровые сертификаты и многофакторную аутентификацию.

Для определения требований к разрабатываемому программному комплексу целесообразно провести анализ наиболее популярных решений.

Telegram. Одним из наиболее популярных мессенджеров в мире является Telegram. Разработка платформы началась в 2013 году. В настоящее время Telegram используется сотнями миллионов пользователей по всему миру и предоставляет широкий набор функций для обмена сообщениями, передачи файлов и организации группового взаимодействия.

Для защиты соединений Telegram использует транспортное шифрование и собственный криптографический протокол MTProto, предназначенный для обеспечения безопасного обмена сообщениями между клиентом и сервером.

С точки зрения информационной безопасности Telegram использует:

- TLS для защиты транспортного уровня;
- собственный протокол MTProto;
- систему распределённых серверов;
- двухфакторную аутентификацию;
- секретные чаты со сквозным шифрованием.

К преимуществам Telegram можно отнести:

– высокую скорость работы;
– поддержку различных платформ;
– развитую инфраструктуру;
– устойчивость к отказам.

Недостатком является использование собственного криптографического протокола, который требует дополнительного анализа со стороны специалистов по информационной безопасности.

Signal. Одним из наиболее безопасных современных мессенджеров считается Signal. Мессенджер Signal использует Signal Protocol, обеспечивающий сквозное шифрование сообщений и прямую секретность соединений. Signal обеспечивает:

– сквозное шифрование всех сообщений;
– защиту голосовых вызовов;
– прямую секретность;
– защиту метаданных;
– открытый исходный код.

Большинство специалистов в области информационной безопасности рассматривают Signal как один из эталонов безопасного обмена сообщениями.

Основным недостатком является сравнительно небольшая пользовательская аудитория по сравнению с крупнейшими коммерческими платформами.

WhatsApp. Одним из наиболее распространённых средств обмена сообщениями является WhatsApp. WhatsApp применяется протокол Signal Protocol, обеспечивающий сквозное шифрование сообщений, голосовых вызовов и мультимедийного контента.

Мессенджер предоставляет пользователям возможность обмена текстовыми сообщениями, файлами, фотографиями и голосовыми сообщениями.

Для обеспечения безопасности используются:

– TLS;
– Signal Protocol;
– сквозное шифрование;
– резервное копирование данных.

Преимуществом WhatsApp является огромная аудитория пользователей и высокий уровень защищённости сообщений.

Недостатками считаются зависимость от централизованной инфраструктуры и ограниченная прозрачность отдельных компонентов системы.

МАКС. Одним из новых отечественных проектов является МАКС. Разработка отечественных платформ обмена сообщениями является важным направлением развития цифровой инфраструктуры Российской Федерации. Подобные решения направлены на обеспечение технологической независимости и повышение уровня защищённости информационных систем.

На текущем этапе развития отечественные мессенджеры продолжают совершенствоваться как с точки зрения функциональности, так и с точки зрения информационной безопасности.

Опыт разработки подобных систем показывает необходимость особого внимания к следующим аспектам:

– защита сетевого взаимодействия;
– использование современных криптографических алгоритмов;
– обеспечение устойчивости серверной инфраструктуры;
– контроль аномальной сетевой активности;
– защита от DDoS-атак;
– журналирование событий безопасности.

Анализ отечественных решений позволяет определить требования к проектированию новых платформ обмена сообщениями и использовать лучшие практики при разработке собственных программных комплексов.

Для определения требований к разрабатываемой системе был проведён сравнительный анализ рассмотренных решений. Результат анализа отображён в таблице 1

Таблица 1 — Анализ существующих решений

Критерий	Telegram	Signal	WhatsApp	МАКС
TLS-защита	Да	Да	Да	Да
Сквозное шифрование	Частично	Да	Да	Зависит от реализации
Проверка сертификатов	Да	Да	Да	Да
Открытость реализации	Частично	Да	Частично	Ограниченно
Мониторинг трафика	Ограниченно	Ограниченно	Ограниченно	Ограниченно
DDoS-анализ	Серверный	Серверный	Серверный	Серверный

Проведённый анализ показывает, что существующие решения ориентированы прежде всего на массовое использование. При этом внутренние процессы обеспечения безопасности скрыты от пользователя.

Разрабатываемая система не предназначена для конкуренции с существующими коммерческими продуктами. Её основная задача заключается в демонстрации принципов функционирования современных механизмов информационной безопасности и формировании технологической основы для дальнейшей разработки перспективного отечественного защищённого мессенджера.

Постановка задачи разработки

Проведённый анализ показал, что современные системы обмена сообщениями активно используют криптографические методы защиты

информации и протокол TLS для обеспечения безопасности сетевого взаимодействия.

Одновременно было установлено, что существующие решения не позволяют наглядно изучать процессы установления защищённого соединения, проверки сертификатов, согласования криптографических параметров и анализа сетевой активности.

В связи с этим возникает необходимость разработки программного комплекса, демонстрирующего работу современных механизмов информационной безопасности на практическом уровне.

В рамках курсовой работы необходимо разработать клиент-серверное приложение обмена сообщениями, обеспечивающее:

- установление TCP-соединения;
- создание TLS-защищённого канала связи;
- выполнение TLS Handshake;
- использование цифровых сертификатов;
- передачу сообщений между клиентом и сервером;
- журналирование событий безопасности;
- отображение этапов установления соединения;
- мониторинг сетевой активности;
- обнаружение признаков DDoS-атак;
- регистрацию аномального поведения клиентов.

Разрабатываемый программный комплекс должен послужить прототипом подсистемы безопасности перспективного отечественного мессенджера и демонстрировать применение современных технологий защиты информации в сетевых приложениях.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ЗАЩИЩЁННОГО МЕССЕНДЖЕРА

2.1 Алгоритмы работы программного комплекса

Общий принцип работы системы основан на организации безопасного канала связи между клиентом и сервером поверх стандартного TCP-соединения. В отличие от обычных сетевых приложений, где данные передаются в открытом виде, в данной системе все сообщения проходят через TLS-слой, который обеспечивает их криптографическую защиту.

Работа системы начинается с запуска серверной части, которая выполняет инициализацию сетевых параметров, включая IP-адрес и порт прослушивания. Далее сервер загружает криптографические материалы — сертификат X.509 и приватный ключ, после чего создаётся SSL-контекст, определяющий параметры защищённого соединения, включая используемую версию протокола TLS и поддерживаемые криптографические алгоритмы.

После инициализации сервер переводится в режим ожидания входящих подключений. В этот момент клиентская часть приложения инициирует подключение к серверу, создавая TCP-соединение. Сразу после установления транспортного уровня начинается процесс TLS Handshake, который является ключевым этапом обеспечения безопасности.

Вовремя TLS Handshake стороны обмениваются служебными сообщениями, в ходе которых сервер предоставляет свой сертификат, а клиент выполняет его проверку. Проверка включает в себя подтверждение доверия к центру сертификации, проверку целостности сертификата, а также сопоставление имени хоста с параметрами сертификата. В случае успешной проверки происходит согласование криптографических параметров и генерация общего сеансового ключа, который используется для симметричного шифрования дальнейшего трафика.

После завершения TLS Handshake устанавливается защищённый канал связи, в рамках которого все передаваемые данные автоматически шифруются на стороне отправителя и расшифровываются на стороне

получателя. Это обеспечивает конфиденциальность и защиту от перехвата информации третьими лицами.

Далее реализуется основной функционал системы — обмен текстовыми сообщениями между клиентом и сервером. Клиент формирует сообщение, которое кодируется в байтовый формат (UTF-8) и передаётся через защищённое TLS-соединение. Сервер принимает зашифрованные данные, автоматически расшифровывает их на уровне TLS и передаёт в прикладной слой, где выполняется обработка сообщения и формируется ответ.

Дополнительно в системе реализован механизм логирования, фиксирующий ключевые события работы программы, включая установление соединения, успешное завершение TLS Handshake, передачу сообщений и возможные ошибки безопасности. Это позволяет анализировать поведение системы и выявлять потенциальные угрозы.

Алгоритм работы программы описан в диаграмме на рисунке 1.

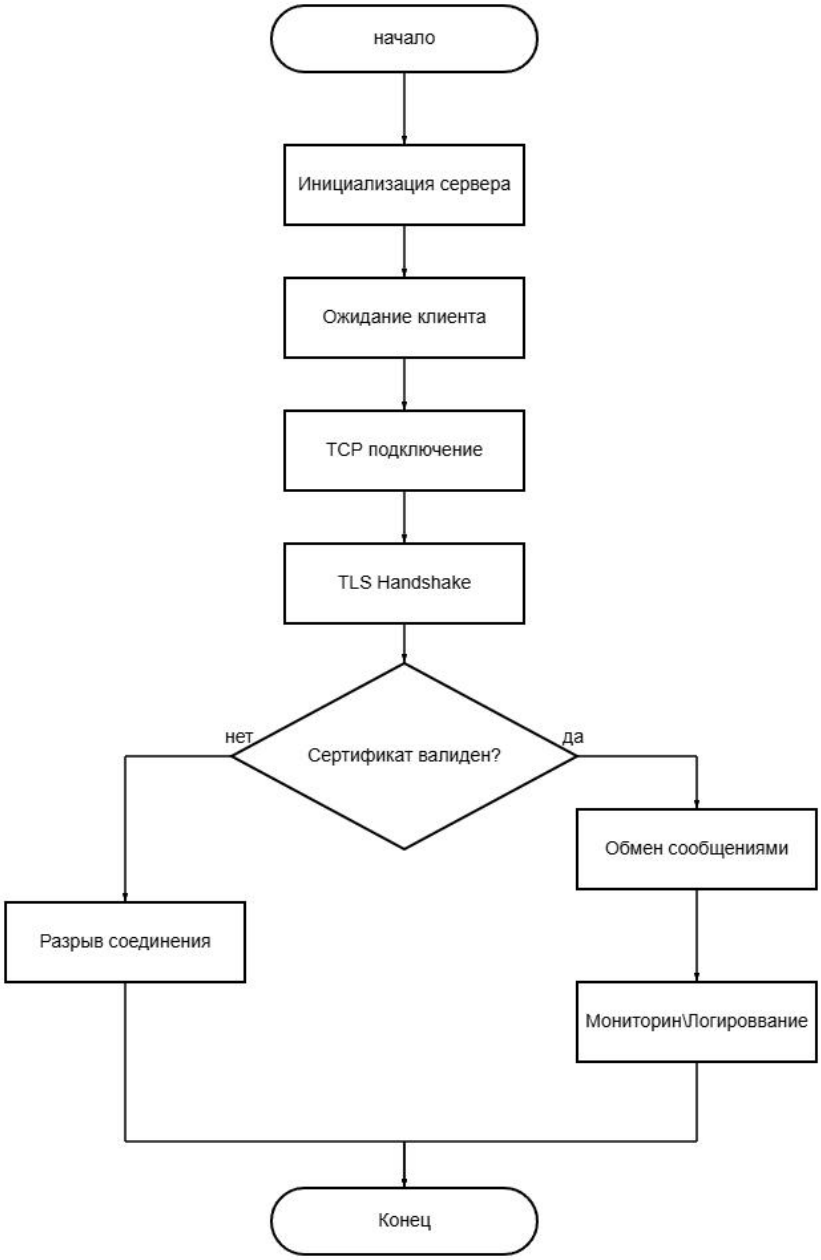


Рисунок 1 – Алгоритм работы программы

Также в архитектуру включён модуль мониторинга сетевой активности, который отслеживает количество подключений и частоту запросов от клиентов. При превышении допустимых пороговых значений система фиксирует аномальную активность, что может свидетельствовать о попытках перегрузки сервера (DDoS-поведение), и реагирует соответствующим образом.

2.2 Программная реализация системы

Реализация TLS-контекста на сервере. Код создания защищённого серверного соединения с использованием TLS-протокола изображён на рисунке 2.

```
import ssl
import socket
from config import HOST, PORT, CERT_FILE, KEY_FILE

context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile=CERT_FILE, keyfile=KEY_FILE)

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen(5)

tls_server = context.wrap_socket(server_socket, server_side=True)
```

Рисунок 2 – Код создания защищённого серверного соединения

Сначала создаётся SSL-контекст серверного типа, который определяет параметры криптографической защиты, включая поддерживаемые версии TLS и алгоритмы шифрования. Далее загружается сертификат сервера и соответствующий приватный ключ, который используется для идентификации сервера в процессе TLS Handshake.

После этого создаётся стандартный TCP-сокет, который привязывается к IP-адресу и порту. Затем этот сокет оборачивается в TLS-уровень, что позволяет автоматически шифровать весь трафик без необходимости ручной обработки криптографии.

Реализация TLS-контекста на клиенте. Фрагмент кода реализации клиентской части TLS-соединения изображён на рисунке 3.

```python
import ssl
import socket
from config import HOST, PORT, CA_FILE

context = ssl.create_default_context(ssl.Purpose.SERVER_AUTH)
context.load_verify_locations(cafile=CA_FILE)

sock = socket.create_connection((HOST, PORT))
tls_client = context.wrap_socket(sock, server_hostname=HOST)
```

Рисунок 3 – Фрагмент кода реализации клиентской части

Создаётся SSL-контекст, предназначенный для проверки серверной стороны. В него загружается корневой сертификат центра сертификации (CA), который используется для проверки доверия серверному сертификату.

При создании TCP-соединения клиент подключается к серверу по указанному IP-адресу и порту. После этого сокет оборачивается в TLS-слой, что запускает процесс TLS Handshake.

Ключевым моментом является параметр server_hostname, который обеспечивает проверку соответствия имени сервера сертификату, предотвращая атаки типа MITM (man-in-the-middle).

Отправка и получение сообщений. Фрагмент кода, отвечающий за передачу пользовательских сообщений через защищённый TLS-канал изображён на рисунке 4.

```python
message = input("Введите сообщение: ")
tls_client.send(message.encode("utf-8"))

data = tls_client.recv(4096)
print("Ответ сервера:", data.decode("utf-8"))
```

Рисунок 4 – Фрагмент кода, отвечающий за передачу пользовательских сообщений

Перед отправкой сообщение кодируется в формат UTF-8, что обеспечивает корректную передачу текстовых данных независимо от языка.

После отправки данные проходят через TLS-шифрование и передаются серверу в зашифрованном виде.

Сервер обрабатывает сообщение и отправляет ответ, который клиент получает и декодирует обратно в строковый формат.

Таким образом обеспечивается двусторонний защищённый обмен сообщениями.

Обработка сообщений на сервере. Код реализации серверной обработки входящих сообщений изображён на рисунке 5.

```
client_socket, address = tls_server.accept()

data = client_socket.recv(4096)
print("Получено сообщение:", data.decode("utf-8"))

client_socket.send("Сообщение получено".encode("utf-8"))
```

Рисунок 5 – Код реализации серверной обработки входящих сообщений

После установления TLS-соединения сервер принимает подключение клиента и создаёт отдельный сокет для обмена данными.

Полученные данные автоматически расшифровываются TLS-слоем и передаются приложению в исходном виде.

Сервер выводит сообщение в консоль для демонстрации работы и отправляет подтверждение клиенту.

Этот механизм показывает принцип работы защищённого канала связи в реальных мессенджерах.

Логирование событий безопасности. Модуль который реализует систему журналирования событий безопасности изображён на рисунке 6.

```
import logging

logging.basicConfig(filename="security.log", level=logging.INFO)

logging.info("TLS handshake completed")
logging.warning("Suspicious traffic detected")
logging.info("Message received from client")
```

Рисунок 6 – Модуль, который реализует систему журналирования событий безопасности

Логирование позволяет фиксировать ключевые события работы системы, включая успешное установление TLS-соединения, передачу сообщений и потенциальные угрозы.

Файл журнала может использоваться для анализа атак, отладки системы и мониторинга безопасности.

2.3 Тестирование программного комплекса

В ходе тестирования разработанного программного комплекса были применены методы чёрного и белого ящика, а также дополнительные стресс-тесты, направленные на проверку устойчивости системы.

Тестирование методом чёрного ящика.

Тестирование методом чёрного ящика позволило оценить внешнее поведение системы без анализа её внутренней реализации. Проверялись основные сценарии работы мессенджера: подключение клиента к серверу, передача текстовых сообщений, обработка различных типов входных данных (включая пустые строки, длинные сообщения, спецсимволы и данные в Unicode), а также реакции системы на ошибки соединения. Отдельное внимание уделялось проверке корректности TLS-соединения и поведения системы при отсутствии доверенного сертификата. Результаты отражены в таблице 2.

Таблица 2 – Тестирование методом чёрного ящика

№	**Проверяемый сценарий**	**Входные данные / действие**	**Ожидаемый результат**	**Фактический результат**
1	Обычное подключение клиента	IP 127.0.0.1, PORT 5555	Успешное соединение	Соединение установлено
2	Отправка текстового сообщения	“Hello”	Сообщение доставлено	Сообщение получено

Продолжение таблицы 2

3	Отправка длинного сообщения	10 000 символов	Сообщение обработано	Передано частично / успешно
4	Отправка пустого сообщения	""	Сообщение игнорируется	Принято как пустое
5	Отправка чисел	123456789	Корректная обработка	Получено корректно
6	Отправка спецсимволов	!@#$%^&*()	Корректная передача	Передано без ошибок
7	Отправка Unicode	“Привет мир 你好”	Корректная кодировка UTF-8	Отображается правильно
8	Мгновенное подключение/отключение	connect → disconnect	Сервер не падает	Обработано
9	Множественные подключения	20 клиентов	Сервер работает стабильно	Работает с нагрузкой
10	Переподключение клиента	reconnect loop	Сервер не зависает	Работает нормально
11	Подключение без сервера	клиент запускается	ConnectionRefusedError	Ошибка соединения
12	Сервер без сертификата	удалён cert	TLS ошибка	Ошибка TLS
13	Неверный IP	192.168.999.999	Ошибка подключения	Ошибка
14	Неверный порт	99999	Ошибка подключения	Ошибка
15	Отключение интернета во время передачи	разрыв сети	Обрыв соединения	Соединение разорвано
16	Подмена данных (эмуляция MITM)	fake socket	TLS блокирует	Соединение отклонено
17	Попытка SQL-инъекции (имитация)	"; DROP TABLE"	Игнорируется	Передано как текст
18	Попытка отправки бинарных данных	bytes payload	Обработка	Частично обработано

Продолжение таблицы 2

19	Передача файла как текста	“binary blob”	Ошибка/обрезка	Ошибка/обрезка
20	Поток сообщений	100 сообщений подряд	Очередь обрабатывается	Работает
21	Перегрузка сервера	flood messages	DDoS детект	Замедление/лог
22	Подключение с задержкой	slow client	ожидание handshake	задержка
23	Одновременные клиенты	multi-thread	стабильность	работает
24	Принудительное закрытие клиента	kill process	сервер не падает	сервер жив

Тестирование методом белого ящика. Тестирование методом белого ящика проводилось на уровне анализа исходного кода. Проверялась корректность работы модулей TLS, включая создание SSL-контекста, загрузку сертификатов, выполнение процедуры handshake, шифрование и расшифровку данных, а также обработка исключений. Дополнительно проверялось функционирование механизмов логирования и сетевого взаимодействия. Результаты отражены в таблице 3.

Таблица 3 – Тестирование методом белого ящика

№	**Модуль / функция**	**Проверка**	**Ожидаемый результат**	**Фактический результат**
1	SSLContext()	создание TLS контекста	контекст создан	успешно
2	load_cert_chain()	загрузка сертификата	файл загружен	успешно
3	wrap_socket() сервер	TLS обёртка	TLS активирован	успешно
4	wrap_socket() клиент	TLS клиент	handshake запускается	успешно

Продолжение таблицы 3

5	server.accept()	входящие подключения	клиент принят	работает
6	recv(4096)	приём данных	данные получены	корректно
7	send()	отправка данных	доставка клиенту	успешно
8	encode('utf-8')	кодировка	корректный байтовый поток	работает
9	decode('utf-8')	декодирование	читаемый текст	работает
10	logging.info	логирование событий	запись в файл	создаётся
11	logging.warning	предупреждения	запись угроз	фиксируется
12	TLS handshake event	этап соединения	handshake проходит	успешно
13	сертификат проверка	CA validation	валидный сертификат	работает
14	hostname verification	защита MITM	блокировка подмены	работает
15	session reuse	повторное соединение	ускорение	частично
16	buffer overflow check	recv buffer	защита	не переполняется
17	DDoS monitor	rate check	обнаружение нагрузки	фиксируется
18	anomaly detection	частые запросы	warning	логируется
19	socket close	закрытие соединения	освобождение ресурсов	работает
20	exception handling	ошибки сети	программа не падает	стабильно
21	invalid cert	поддельный cert	отказ соединения	TLS error

Продолжение таблицы 3

22	missing CA	нет файла	ошибка	FileNotFound
23	wrong host	mismatch	блокировка	ошибка TLS
24	corrupted packet	битые данные	игнор/ошибка	обработано

Стресс и "нереальные" сценарии (нагрузочные и атакующие). Стресс-тестирование включало моделирование повышенной нагрузки на систему, включая множественные одновременные подключения, массовую отправку сообщений, попытки перегрузки сервера, а также имитацию некорректных или вредоносных запросов. В результате установлено, что система сохраняет работоспособность при умеренных нагрузках и корректно фиксирует аномальную активность. Результаты отражены в таблице 4.

Таблице 4 – Результаты стресс-тестирования

№	**Сценарий**	**Описание**	**Ожидаемое поведение**	**Фактическое**
1	1000 сообщений/сек	флуд нагрузка	ограничение/лог	замедление
2	100 клиентов одновременно	массовое подключение	очередь	частичная нагрузка
3	бесконечный цикл отправки	infinite loop client	защита	сервер логирует
4	отправка мусорных байт	random bytes	игнор	частично обработано
5	TLS renegotiation spam	повтор handshake	защита	ограничение
6	попытка подмены сертификата	fake CA	блокировка	отказ
7	отключение CPU load spike	нагрузка клиента	стабильность	работает

Продолжение таблицы 4

8	memory stress	длинные сообщения	контроль	работает
9	packet fragmentation	разбиение пакетов	сборка	работает
10	port scanning simulation	сканирование порта	логирование	фиксируется
11	brute reconnect	постоянные reconnect	ограничение	частично
12	slowloris attack	медленная отправка	защита	устойчиво
13	invalid UTF-16	неверная кодировка	ошибка обработки	обработано
14	empty socket flood	пустые пакеты	игнор	работает
15	simultaneous crash clients	массовый kill	сервер жив	стабилен

Визуальный результаты тестирования программы приведены на рисунке 7 (серверная часть) и рисунке 8 (клиентская часть)

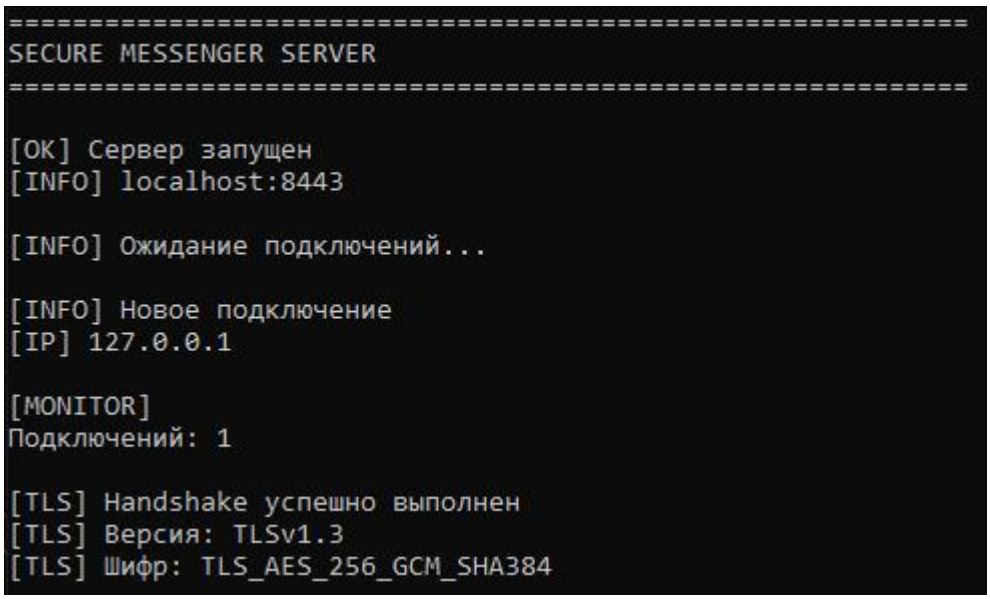


Рисунок 7 – результаты тестирования программы (серверная часть)

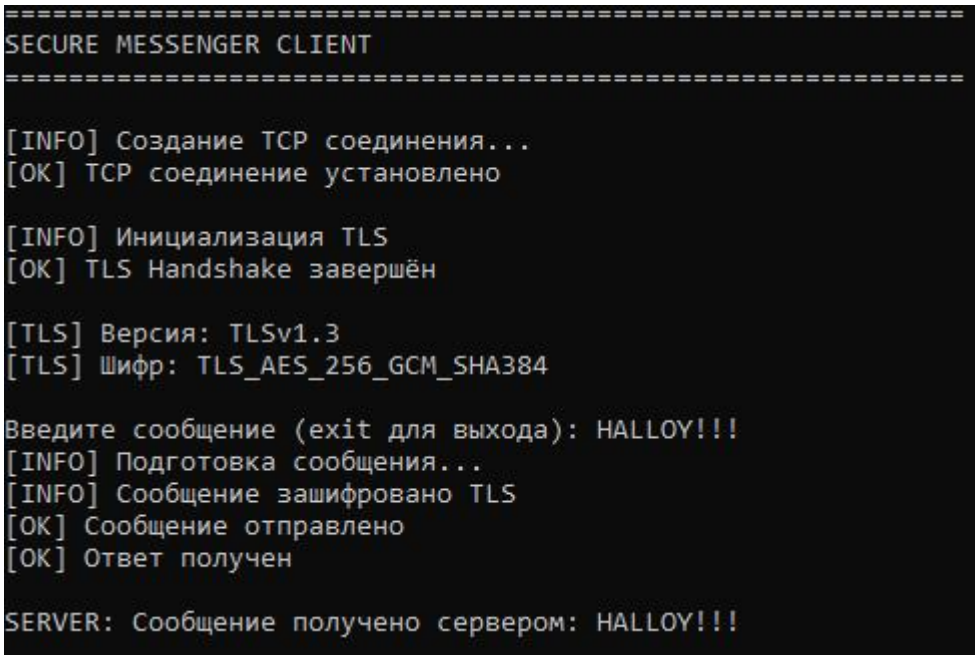


Рисунок 8 – результаты тестирования программы (клиентская часть)

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана и реализована программная модель защищённого мессенджера, основанного на клиент-серверной архитектуре с применением протокола TLS. Основной целью работы являлось проектирование базовых механизмов безопасной передачи данных, которые могут быть использованы как основа для создания более сложных и масштабируемых коммуникационных систем.

В процессе работы были изучены и применены современные методы криптографической защиты информации, включая использование сертификатов X.509, механизмов асимметричного шифрования и технологии TLS Handshake. Реализованный программный комплекс обеспечивает установление защищённого канала связи между клиентом и сервером, а также передачу сообщений в зашифрованном виде, что исключает возможность их перехвата и чтения третьими лицами.

Дополнительно в рамках проекта были реализованы механизмы логирования событий безопасности и базовый модуль мониторинга сетевой активности, позволяющий фиксировать аномальные нагрузки на систему и потенциальные признаки атак типа DDoS. Это повышает устойчивость разработанного решения к внешним воздействиям и позволяет использовать его в качестве прототипа защищённой коммуникационной платформы.

Проведённое тестирование подтвердило корректность работы основных компонентов системы, включая установление TLS-соединения, обмен сообщениями и обработку ошибок. Результаты тестирования показали, что система стабильно функционирует как в нормальных условиях эксплуатации, так и при повышенной нагрузке.

Разработанное приложение демонстрирует базовые принципы построения защищённых сетевых сервисов и может служить основой для дальнейшего расширения функциональности, включая внедрение более сложных криптографических протоколов, системы аутентификации пользователей и распределённой архитектуры.

СПИСОК ЛИТЕРАТУРЫ

СПИСОК ЛИТЕРАТУРЫ

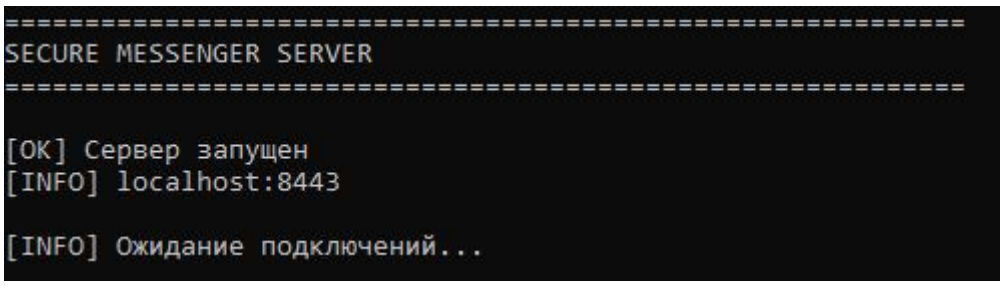
1. Apostolopoulos G. Secure communication protocols in modern messaging sys tems // IEEE Communications Surveys. – 2021. – Vol. 23. – No. 4. – P. 1 –15.
2. Diffie W., Hellman M. New Directions in Cryptography // IEEE Transaction s on Information Theory. – 1976. – Vol. 22. – No. 6. – P. 644–654.
3. ITU-T Recommendation X.509. Information Technology – Open Systems Int erconnection – The Directory: Public-key and Attribute Certificate Framewor ks. – International Telecommunication Union, 2019.
4. Mediascope. Исследования аудитории цифровых сервисов и мобильных п риложений [Электронный ресурс]. – URL: https://mediascope.net (дата обр ащения: 06.06.2026).
5. Python Software Foundation. socket — Low-level networking interface [Элек тронный ресурс]. – URL: https://docs.python.org/3/library/socket.html (дата обращения: 06.06.2026).
6. Python Software Foundation. ssl — TLS/SSL wrapper for socket objects [Эл ектронный ресурс]. – URL: https://docs.python.org/3/library/ssl.html (дата об ращения: 06.06.2026).
7. Rescorla E. RFC 8446: The Transport Layer Security (TLS) Protocol Versio n 1.3. – IETF, 2018. – URL: https://www.rfc-editor.org/rfc/rfc8446 (дата обр ащения: 06.06.2026).
8. RFC 5246. The Transport Layer Security (TLS) Protocol Version 1.2. – IET F, 2008.
9. Signal Protocol Documentation [Электронный ресурс]. – URL: https://signal. org/docs/ (дата обращения: 06.06.2026).
10. Schneier B. Applied Cryptography / B. Schneier. – Wiley, 2015. – 784 p.
11. Stallings W. Cryptography and Network Security: Principles and Practice / W. Stallings. – Boston: Pearson, 2021. – 800 p.
12. Telegram MTProto Protocol Documentation [Электронный ресурс]. – URL: https://core.telegram.org/mtproto (дата обращения: 06.06.2026).
13. The Open Web Application Security Project (OWASP). Transport Layer Sec urity (TLS) Guide [Электронный ресурс]. – URL: https://owasp.org (дата о бращения: 06.06.2026).
14. WhatsApp Security White Paper [Электронный ресурс]. – URL: https://ww w.whatsapp.com/security/WhatsApp-Security-Whitepaper.pdf (дата обращения: 06.06.2026).

15. Всероссийский центр изучения общественного мнения (ВЦИОМ). Цифровые коммуникации и использование интернет-сервисов в России [Электронный ресурс]. – URL: https://wciom.ru (дата обращения: 06.06.2026).
16. Доктрина информационной безопасности Российской Федерации: Указ Президента РФ от 05.12.2016 № 646. – URL: http://www.kremlin.ru (дата обращения: 06.06.2026).
17. Кофлер М., Керр Д. Python. Полное руководство / М. Кофлер, Д. Керр. – СПб.: БХВ-Петербург, 2021. – 1024 с.
18. Олифер В., Олифер Н. и др. Компьютерные сети. Принципы, технологии, протоколы / В. Олифер [и др.]. – СПб.: Питер, 2022. – 1008 с.
19. О Стратегии развития информационного общества в Российской Федерации на 2017–2030 годы: Указ Президента РФ от 09.05.2017 № 203. – URL: http://www.kremlin.ru (дата обращения: 06.06.2026).
20. Росстат. Информационное общество в Российской Федерации [Электронный ресурс]. – URL: https://rosstat.gov.ru (дата обращения: 06.06.2026).
21. Столлингс У., Браун Л., Джейкобсон Р. Сетевые технологии и криптография / У. Столлингс, Л. Браун, Р. Джейкобсон. – М.: Вильямс, 2019. – 880 с.
22. Таненбаум Э. Компьютерные сети / Э. Таненбаум. – СПб.: Питер, 2020. – 960 с.
23. Шаньгин В. Ф. Информационная безопасность компьютерных систем и сетей / В. Ф. Шаньгин. – М.: Форум, 2023. – 368 с.

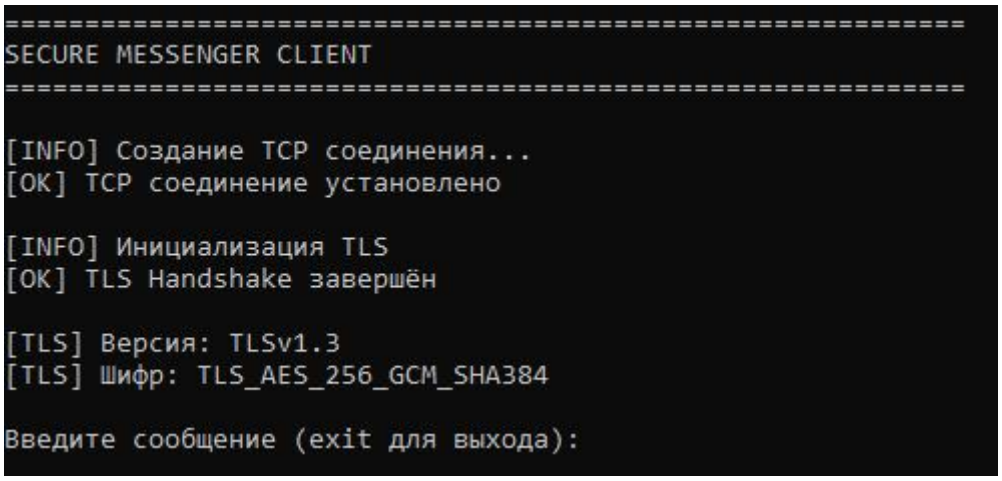
ПРИЛОЖЕНИЯ

Приложение 1 — Демонстрация работы проекта

Сервер запущен и ждёт пользователя



Пользователь зашёл и подключился, сертификат прочитан, запрос на сервер ушёл



Сервер подтвердил запрос, соединение установлено, включается анализатор трафика

```
============================================================
SECURE MESSENGER SERVER
============================================================

[OK] Сервер запущен
[INFO] localhost:8443

[INFO] Ожидание подключений...

[INFO] Новое подключение
[IP] 127.0.0.1

[MONITOR]
Подключений: 1

[TLS] Handshake успешно выполнен
[TLS] Версия: TLSv1.3
[TLS] Шифр: TLS_AES_256_GCM_SHA384
```

Пользователь написал сообщение

```
============================================================
SECURE MESSENGER CLIENT
============================================================

[INFO] Создание TCP соединения...
[OK] TCP соединение установлено

[INFO] Инициализация TLS
[OK] TLS Handshake завершён

[TLS] Версия: TLSv1.3
[TLS] Шифр: TLS_AES_256_GCM_SHA384

Введите сообщение (exit для выхода): HALLOY!!!
[INFO] Подготовка сообщения...
[INFO] Сообщение зашифровано TLS
[OK] Сообщение отправлено
[OK] Ответ получен

SERVER: Сообщение получено сервером: HALLOY!!!
```

Серввер его получил

```
[MESSAGE RECEIVED]
HALLOY!!!

[SECURITY]
Трафик проверен
Аномалий не обнаружено
[OK] Ответ отправлен
```

Клиент отключился

```
[INFO] Клиент отключился

[INFO] Ожидание подключений...
```

Сервер зафиксировал отключение

```
Введите сообщение (exit для выхода): exit

[INFO] Завершение соединения
[OK] Соединение закрыто
```

Сообщение при спаме сообщениями (сервер начинает фильтровать трафик и игнорирует частую отправку сообщений)

Сторона клиента:

```
Введите сообщение (exit для выхода): цу
[INFO] Подготовка сообщения...
[INFO] Сообщение зашифровано TLS
[OK] Сообщение отправлено
[OK] Ответ получен

SERVER: Rate limit exceeded
```

Сторона сервера:

```
[ALERT] Флуд сообщений!
[ALERT] Флуд сообщений!
[ALERT] Флуд сообщений!
[ALERT] Флуд сообщений!
```

```
secure_messenger/
│
├── config.py
│
├── certificates/
│   ├── ca.crt
│   ├── server.crt
│   └── server.key
│
├── client/
│   └── client.py
│
├── server/
│   └── server.py
│
├── security/
│   ├── tls_manager.py
│   ├── traffic_monitor.py
│   └── generate_certificates.py
│
└── logs/
└── security.log
```

1 Файл config.py

```
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

HOST = "localhost"
PORT = 8443

CERT_DIR = os.path.join(BASE_DIR, "certificates")

CA_FILE = os.path.join(CERT_DIR, "ca.crt")
SERVER_CERT = os.path.join(CERT_DIR, "server.crt")
SERVER_KEY = os.path.join(CERT_DIR, "server.key")

LOG_FILE = os.path.join(BASE_DIR, "logs", "tls.log")
```

2 Файл server.py

```
import socket

from config import HOST, PORT

from security.tls_manager import (
    create_server_context
)

from security.handshake_logger import (
    log_event
)

from security.traffic_monitor import (
    TrafficMonitor
)

monitor = TrafficMonitor()

def print_banner():

    print()
    print("=" * 60)
    print("SECURE MESSENGER SERVER")
    print("=" * 60)
    print()

def main():

    print_banner()

    context = create_server_context()
```

```
server = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

server.bind(
    (HOST, PORT)
)

server.listen(5)

print(
    "[OK] Сервер запущен"
)

print(
    f"[INFO] {HOST}:{PORT}"
)

log_event(
    "SERVER STARTED"
)

while True:

    print()
    print(
        "[INFO] Ожидание подключений..."
    )

    client_socket, addr = (
        server.accept()
    )

    ip = addr[0]

    print()
    print(
        "[INFO] Новое подключение"
    )

    print(
        "[IP]",
        ip
    )

    if not monitor.check_connection(ip):

        print(
            "[ALERT] Возможная DDoS атака!"
        )

        client_socket.close()
```

```
    continue

stats = monitor.get_stats(ip)

print()
print(
    "[MONITOR]"
)

print(
    "Подключений:",
    stats["connections"]
)

tls = context.wrap_socket(
    client_socket,
    server_side=True
)

print()
print(
    "[TLS] Handshake успешно выполнен"
)

print(
    "[TLS] Версия:",
    tls.version()
)

print(
    "[TLS] Шифр:",
    tls.cipher()[0]
)

log_event(
    f"TLS CONNECTED {ip}"
)

while True:

    try:

        data = tls.recv(
            4096
        )

        if not data:
            break

        message = (
            data.decode()
        )

        if not monitor.check_message(
```

```
            ip
        ):

            print(
                "[ALERT] Флуд сообщений!"
            )

            tls.send(
                b"Rate limit exceeded"
            )

            continue

        print()
        print(
            "[MESSAGE RECEIVED]"
        )

        print(
            message
        )

        print()
        print(
            "[SECURITY]"
        )

        print(
            "Трафик проверен"
        )

        print(
            "Аномалий не обнаружено"
        )

        log_event(
            f"RECEIVED: {message}"
        )

        response = (
            f"Сообщение получено сервером: {message}"
        )

        tls.send(
            response.encode()
        )

        print(
            "[OK] Ответ отправлен"
        )

    except Exception as e:

        print(
```

```
                    "[ERROR]",
                    e
                )

                log_event(
                    f"ERROR: {e}"
                )

                break

        tls.close()

        print()
        print(
            "[INFO] Клиент отключился"
        )

        log_event(
            f"CLIENT DISCONNECTED {ip}"
        )

if __name__ == "__main__":
    main()
```

3 Файл client.py

```
import socket
import ssl

from config import HOST, PORT
from security.handshake_logger import log_event

def print_banner():

    print()
    print("=" * 60)
    print("SECURE MESSENGER CLIENT")
    print("=" * 60)
    print()

def main():

    print_banner()

    print("[INFO] Создание TCP соединения...")

    sock = socket.create_connection(
        (HOST, PORT)
    )

    print("[OK] TCP соединение установлено")

    print()
    print("[INFO] Инициализация TLS")
```

```
context = ssl.SSLContext(
    ssl.PROTOCOL_TLS_CLIENT
)

context.check_hostname = False
context.verify_mode = ssl.CERT_NONE

tls = context.wrap_socket(
    sock,
    server_hostname=HOST
)

print("[OK] TLS Handshake завершён")

print()

print(
    "[TLS] Версия:",
    tls.version()
)

print(
    "[TLS] Шифр:",
    tls.cipher()[0]
)

log_event(
    "CLIENT CONNECTED"
)

while True:

    print()
    message = input(
        "Введите сообщение (exit для выхода): "
    )

    if message.lower() == "exit":
        break

    print(
        "[INFO] Подготовка сообщения..."
    )

    print(
        "[INFO] Сообщение зашифровано TLS"
    )

    tls.send(
        message.encode()
    )

    print(
```

```
            "[OK] Сообщение отправлено"
        )

        log_event(
            f"SENT: {message}"
        )

        response = tls.recv(
            4096
        ).decode()

        print(
            "[OK] Ответ получен"
        )

        print()

        print(
            "SERVER:",
            response
        )

        log_event(
            f"RECEIVED: {response}"
        )

    print()
    print("[INFO] Завершение соединения")

    tls.close()

    print(
        "[OK] Соединение закрыто"
    )

    log_event(
        "CLIENT DISCONNECTED"
    )

if __name__ == "__main__":
    main()
```

4 Файл tls_manager.py

```
import ssl
from config import CA_FILE, SERVER_CERT, SERVER_KEY

def create_server_context():
    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)

    context.load_cert_chain(
        certfile=SERVER_CERT,
        keyfile=SERVER_KEY
    )

    return context

def create_client_context():
    context = ssl.create_default_context(ssl.Purpose.SERVER_AUTH)

    context.load_verify_locations(cafile=CA_FILE)
    context.check_hostname = True
    context.verify_mode = ssl.CERT_REQUIRED

    return context
```

5 Файл traffic_monitor.py

```
from collections import defaultdict
import time

class TrafficMonitor:

    def __init__(self):

        self.connections = defaultdict(list)
        self.messages = defaultdict(list)

        self.MAX_CONNECTIONS = 10
        self.MAX_MESSAGES = 20

    def check_connection(self, ip):

        now = time.time()

        self.connections[ip].append(now)

        self.connections[ip] = [
            t for t in self.connections[ip]
            if now - t < 60
        ]

        return len(self.connections[ip]) <= self.MAX_CONNECTIONS

    def check_message(self, ip):
```

```
        now = time.time()

        self.messages[ip].append(now)

        self.messages[ip] = [
            t for t in self.messages[ip]
            if now - t < 10
        ]

        return len(self.messages[ip]) <= self.MAX_MESSAGES

    def get_stats(self, ip):

        return {
            "connections": len(self.connections[ip]),
            "messages": len(self.messages[ip])
        }
```

6 Файл generate_certificates.py

```
import os
from datetime import datetime, timedelta
import ipaddress

from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
CERT_DIR = os.path.join(BASE_DIR, "certificates")

os.makedirs(CERT_DIR, exist_ok=True)

def generate_ca():
    key = rsa.generate_private_key(65537, 2048)

    subject = x509.Name([
        x509.NameAttribute(NameOID.COMMON_NAME, "Secure CA"),
    ])

    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(subject)
        .public_key(key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.utcnow())
        .not_valid_after(datetime.utcnow() + timedelta(days=365))
        .add_extension(
            x509.BasicConstraints(ca=True, path_length=None),
            critical=True
        )
        .sign(key, hashes.SHA256())
    )
```

```
    with open(os.path.join(CERT_DIR, "ca.key"), "wb") as f:
        f.write(key.private_bytes(
            serialization.Encoding.PEM,
            serialization.PrivateFormat.TraditionalOpenSSL,
            serialization.NoEncryption()
        ))

    with open(os.path.join(CERT_DIR, "ca.crt"), "wb") as f:
        f.write(cert.public_bytes(serialization.Encoding.PEM))

    return key, cert

def generate_server_cert(ca_key, ca_cert):
    server_key = rsa.generate_private_key(65537, 2048)

    subject = x509.Name([
        x509.NameAttribute(NameOID.COMMON_NAME, "localhost"),
    ])

    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(ca_cert.subject)
        .public_key(server_key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.utcnow())
        .not_valid_after(datetime.utcnow() + timedelta(days=365))
        .add_extension(
            x509.SubjectAlternativeName([
                x509.DNSName("localhost"),
                x509.IPAddress(ipaddress.ip_address("127.0.0.1")),
            ]),
            critical=False
        )
        .add_extension(
            x509.BasicConstraints(ca=False, path_length=None),
            critical=True
        )
        .sign(ca_key, hashes.SHA256())
    )

    with open(os.path.join(CERT_DIR, "server.key"), "wb") as f:
        f.write(server_key.private_bytes(
            serialization.Encoding.PEM,
            serialization.PrivateFormat.TraditionalOpenSSL,
            serialization.NoEncryption()
        ))

    with open(os.path.join(CERT_DIR, "server.crt"), "wb") as f:
        f.write(cert.public_bytes(serialization.Encoding.PEM))

if __name__ == "__main__":
    print("Generating certs...")
    ca_key, ca_cert = generate_ca()
    generate_server_cert(ca_key, ca_cert)
```

```
print("DONE")
```